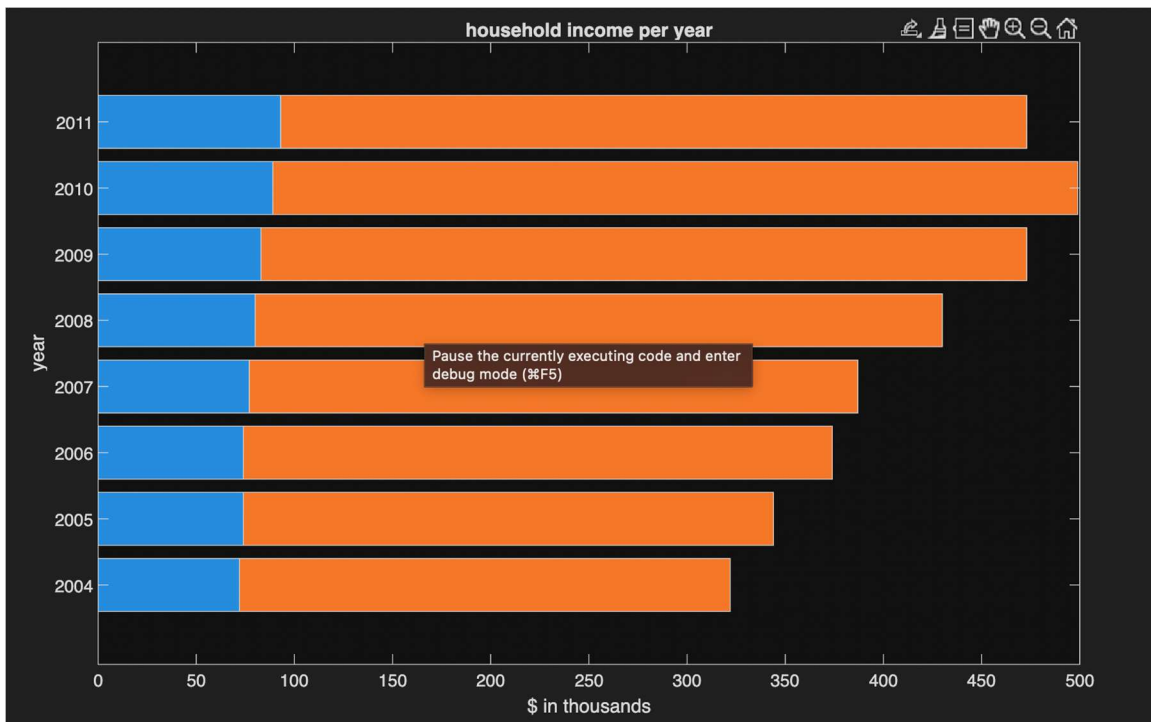


Problem 1

```
clear, clc
%{
Create a file in this format, and then load the information into a matrix. Create a horizontal
stacked bar chart to display the information, with an appropriate title. Note: use the
'XData' property to put the years on the axis as shown in Fig.
%}
load("houseafford.dat")
h = barh(houseafford(2:3,:),'stacked'); %create a horizontal stacked bar chart --
%--that plots rows 2 and 3 of houseafford, transposes data so that years
%are on the y axis
xlabel('$ in thousands') %label the x axis
ylabel('year') %label the y axis
h(1).XData=houseafford(1,:); %force both stacks to use the same x positions.
h(2).XData=houseafford(1,:);
title('household income per year') %title the graph
```

Output :



Problem 8.4

```
clear, clc
%{
Nathaniel Moberg, AERE 1610, Lab 6, Problem 8.4
Purpose: create a cell array containing basic character vectors, then test
and show the difference between different indexing methods in output
%}
sports = {'Basketball', 'Baseball', 'Soccer', 'Football', 'Tennis'}; %initial cell array
sports(1:2) %pulls array positions 1 & 2 as separate array
sports{1:2} %%returns what is located in positions 1 & 2 individually
[p1 p2] = sports{1:2} %pulls cells and assigns them to variables
```

Output :

ans =

1×2 **cell** array

{'Basketball'} {'Baseball'}

ans =

'Basketball'

ans =

'Baseball'

p1 =

'Basketball'

p2 =

'Baseball'

Problem 2

Given a vector of structures defined by the following statements:

```
kit(2) = struct('sub', ...  
struct('id',123,'wt',4.4,'code','a'),...  
'name', 'xyz', 'lens', [4 7])  
kit(1) = struct('sub', ...  
struct('id',33,'wt',11.11,'code','q'), ...  
'name', 'rst', 'lens', 5:6)
```

Which of the following expressions are valid? If the expression is valid, give its value. If it

is not valid, explain why.

```
>> kit(1).sub
```

id: 33

wt: 11.11

code: q

```
>> kit(2).lens(1)
```

4

```
>> kit(1).code
```

Not valid because code is contained within sub so it needs to be called kit(1).sub.code

```
>> kit(2).sub.id == kit(1).sub.id
```

Returns 0 because they are not equal to each other

```
>> strfind(kit(1).name, 's')
```

Returns 2 because kit(1).name is rst and s is the second letter

Problem 6.23

script

```
clear, clc
%{
prompt the user for the user's previous salary and new salary
• call a function to calculate and return the percentage change
• print the resulting percentage increase if it was an increase, or a sympathy message
if not
%}

oldsalary = input('enter previous salary: '); %ask for old salary
newsalary = input('enter new salary: '); %ask for new salary
percentchange = PercentChange(oldsalary,newsalary); %call percent change func
if percentchange > 0 %check if salary increased
fprintf('your salary increased by %g%% ',percentchange)
%print how much it increased by
else
disp('unfortunately your salary did not increase')
%if salary didnt increase disp that
End

-----function

function [percentchange] = PercentChange(oldsalary, newsalary)
%%purpose: return percentage change of salary
arguments (Input)
oldsalary
newsalary
end
arguments (Output)
percentchange
end
percentchange = ((newsalary - oldsalary) / oldsalary) * 100; %calculates percentage change
end
```

Output:

```
enter previous salary: 10
enter new salary: 20
your salary increased by 100% >>
```

Problem 9.7

```
clear, clc
```

```
%{
```

A set of data files named "exfile1.dat", "exfile2.dat", etc. has been created by a series of experiments. It is not known exactly how many there are, but the files are numbered sequentially with integers beginning with 1. The files all store combinations of numbers and characters, and are not in the same format. Write a script that will count how many lines total are in the files. Note that you do not have to process the data in the files in any way; just count the number of lines

```
%}
```

```
totalLines = 0; %initialize total lines
```

```
fileNum = 1; %start at file 1
```

```
while true
```

```
    % Build filename like exfile1.dat, exfile2.dat, ...
    filename = sprintf('exfile%d.dat', fileNum);
```

```
    % Check if file exists
```

```
    if ~isfile(filename)
```

```
        break; % stop when file doesn't exist
```

```
    end
```

```
    % Open file for reading
```

```
    fid = fopen(filename, 'r');
```

```
    % Count lines
```

```
    lineCount = 0; %initialize line count
```

```
    while ~feof(fid) %check if we have reached end of file
```

```
        lineCount = lineCount + 1; %increase linecount for every line in file
```

```
    end
```

```
    fclose(fid); %close the file
```

```
    totalLines = totalLines + lineCount; %add the amount of lines for this file
```

```
    fileNum = fileNum + 1; %increment the file that we are on
```

```
end
```

```
fprintf('Total number of lines across all files: %d\n', totalLines); %print total number of lines
```